Codefest 3

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

Overview

This codefest has two sequential challenges. Complete CMAN before opening CLLM. CMAN must be completed without any AI assistance. CLLM permits and often requires AI tooling. Deliverables are committed to your GitHub repository by the following Sunday at 11:59 pm. Each codefest is graded Satisfactory (S) / Not Yet (NY) as a CMAN+CLLM pair. A late submission counts as Not Yet and consumes one revision token. COPT is an optional problem that allows students to collect an extra token.

CMAN — DRAM traffic analysis: naive vs. tiled matrix multiply

Consider multiplying two square FP32 matrices of size N×N where N=32. Matrix elements are stored and accessed in row-major order.

Tasks

1. Naive triple loop (ijk order): for computing one output element C[i][j] = Σ A[i][k]×B[k][j], how many times is each element of B accessed? Across the full N×N output, how many total element accesses are made to A and B? Compute total DRAM traffic in bytes for the full matrix multiply, assuming every element access goes to DRAM (no data reuse).
2. Tiled loop (tile size T=8): the computation is blocked into T×T tiles. Compute the number of DRAM loads for A and B tiles across the full computation. Compute total DRAM traffic in bytes.
3. Compute the ratio of naive DRAM traffic to tiled DRAM traffic. Explain in one sentence why this ratio equals N.
4. If DRAM bandwidth is 320 GB/s and compute is 10 TFLOPS, compute execution time for the naive case (memory-bound) and the tiled case. For each, state whether the bottleneck is compute or memory.

CMAN NO AI	Deliverable: `codefest/cf03/cman_dram_traffic.md` (or .pdf/.png) containing (a) naive DRAM traffic calculation with formula and values; (b) tiled DRAM traffic calculation; (c) traffic ratio with one-sentence explanation; (d) execution time for naive and tiled cases with bound classification.

GPU and CUDA setup

Checking for a GPU

Before installing CUDA tools, confirm that your machine has a CUDA-capable NVIDIA GPU. On Linux or WSL2, run: `lspci | grep -i nvidia`

On macOS or Windows (outside WSL2), run: `nvidia-smi`

If `nvidia-smi` prints a driver version and a GPU name, you have a working GPU and driver. If neither command finds anything, you do not have a CUDA-capable GPU. In that case, use Google Colab (Runtime → Change runtime type → T4 GPU) or PSU's research computing cluster (`https://orca.pdx.edu`) for the CLLM tasks.

Installing CUDA tools

You need the CUDA Toolkit (which provides `nvcc`) and Nsight Compute (which provides ncu). The CUDA Toolkit version must be compatible with your GPU driver; nvidia-smi reports your driver's maximum supported CUDA version in the top-right corner.

Linux / WSL2 (recommended): Use the CUDA Toolkit installer from `https://developer.nvidia.com/cuda-downloads`. Select your OS, architecture, and distribution.

Choose the deb (network) method for Ubuntu. After installation, add the toolkit to your PATH by appending the following to `~/.bashrc` (or `~/.zshrc`) : `export PATH=/usr/local/cuda/bin:$PATH`
Then run source `~/.bashrc` and verify with `nvcc --version`.
Windows (native): Download the CUDA Toolkit .exe installer from `https://developer.nvidia.com/cuda-downloads`. The installer adds `nvcc` to your PATH automatically. Verify in PowerShell with `nvcc --version`. Note that Nsight Compute is bundled with the toolkit installer on Windows; look for `ncu` in the Nsight Compute installation directory.
Google Colab (no local GPU): CUDA is pre-installed. Run `!nvcc --version` in a code cell to confirm. Upload your .cu files or write them directly in the notebook using `%%writefile`. Compile with `!nvcc -O2 -o gemm_naive gemm_naive.cu` and run with `!./gemm_naive`.

CLLM — CUDA GEMM: naive vs. tiled with roofline placement

Using your favorite LLM, implement and profile two CUDA kernels for 1024×1024 FP32 matrix multiply. All .cu files must compile with `nvcc` without modification.

Tasks

1. Write (or generate with LLM assistance) two CUDA kernels:
   (a) a naive kernel with one thread per output element;
   (b) a shared-memory tiled kernel with tile size 8.
2. Save as `codefest/cf03/cuda/gemm_naive.cu` and `codefest/cf03/cuda/gemm_tiled.cu`.
3. Compile and run both kernels. Measure execution time (use `cudaEventRecord` or `nvprof`). Compute achieved GFLOP/s for each.
4. Profile both kernels with Nsight Compute. Report: achieved memory bandwidth, achieved compute throughput, and roofline position (compute-bound or memory-bound).
5. Plot both kernels on the roofline for your GPU. Annotate with achieved performance and theoretical ceiling. Save as `codefest/cf03/profiling/gemm_roofline.png`.
6. In `codefest/cf03/analysis/gemm_analysis.md` write 150–200 words:
   (a) why the naive kernel is memory-bound;
   (b) how tiling reduces DRAM traffic;
   (c) whether your tiled kernel achieved the expected improvement — if not, identify the remaining bottleneck.

CLLM AI OK	Deliverable: GitHub commit containing `codefest/cf03/cuda/gemm_naive.cu`; `codefest/cf03/cuda/gemm_tiled.cu` (both must compile with `nvcc`); `codefest/cf03/profiling/gemm_roofline.png` with both kernels plotted; `codefest/cf03/analysis/gemm_analysis.md` ≥150 words addressing all 3 sub-questions.

COPT — Optional challenge (+ 1 token): forward pass of a neural network on the GPU

This is an optional challenge for students who finish CMAN and CLLM during the session and want to earn an additional revision token. Completing COPT does not affect your S/NY grade for this codefest. A completed COPT earns 1 revision token, awarded after the instructor reviews the submission. AI assistance is expected.
Implement the forward pass of the network shown in Figure 3 using PyTorch on the GPU. The network has 4 inputs, 5 hidden neurons with ReLU activations, and 1 linear output neuron. Use a batch size of 16 (i.e., process 16 input vectors simultaneously). No training is required; the goal is to set up the network, generate a random input batch, move everything to the GPU, run the forward pass, and verify the output shape.

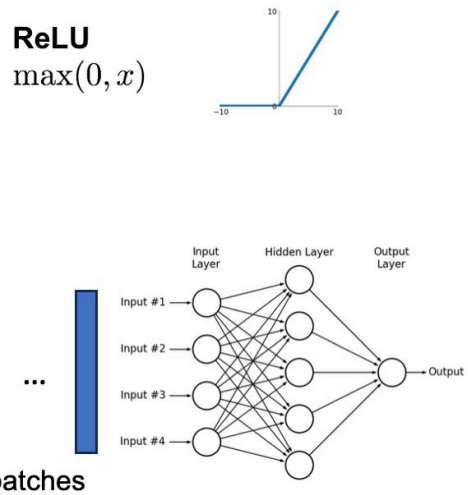

Figure 3. Network architecture for COPT: 4 inputs, 5 hidden ReLU neurons, 1 linear output, batch size 16.

Tasks

1. At the start of the script, detect whether a CUDA GPU is available and print the device name. If no GPU is found, the script should print a clear message and exit without running the training loop. Use: `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`.
2. Define the network using `torch.nn.Sequential` or a custom `nn.Module`: a linear layer 4 → 5, a ReLU activation, and a linear layer 5 → 1 (no activation on the output). Move the model to the GPU with `model.to(device)`.
3. Generate a random input tensor of shape [16, 4] using `torch.randn(16, 4)`. Move it to the GPU. Run the forward pass and print the output tensor shape, which should be [16, 1]. Confirm the output is on the GPU by printing `output.device`.

Save the script as `codefest/cf03/copt/nn_forward_gpu.py`. Include a `codefest/cf03/copt/copt_output.txt` file with the complete terminal output from one successful run.

COPT AI OK	Deliverable: `codefest/cf03/copt/nn_forward_gpu.py` (runs without errors on a CUDA-capable machine); `codefest/cf03/copt/copt_output.txt` (complete terminal output showing device name, output tensor shape [16, 1], and output.device confirming GPU placement).

S/NY criteria

S/NY criteria	**CMAN — S requires:** naive DRAM traffic stated correctly ±10% with formula; tiled traffic stated correctly with formula; ratio = N = 32 stated with one-sentence justification; execution times computed from stated bandwidth/compute values; bound classifications correct (naive memory-bound, tiled likely compute-bound or closer to ridge). **CLLM — S requires:** both .cu files present and compile with `nvcc`; measured GFLOP/s reported for both kernels; Nsight output referenced; roofline plot present with both kernels labeled; gemm_analysis.md ≥150 words addressing all 3 sub-questions. **COPT**: `nn_forward_gpu.py` runs without errors on a CUDA device and `copt_output.txt` shows device name, output tensor shape [16, 1], and output.device confirming GPU placement; earns 1 revision token. **NY if:** either .cu file does not compile; roofline plot absent; analysis <75 words.